Prolog Programming Laboratory

Name - Prachi Ganesh Lohkare

PRN - 202401040025

Batch - A1

Roll No. - 04

CA ACTIVITY - 1B

Title

Design and Implementation of a Knowledge-Based System in Prolog for a Harry Potter Rule-Based Magical Advisor

Aim

To study the fundamentals of logic programming using Prolog and implement a knowledge base containing facts and rules about Harry Potter characters, their houses, skills, objects, and situations, and to derive new information through rule-based inference.

Theory

Prolog (Programming in Logic) is a declarative programming language used in Artificial Intelligence and expert systems. Instead of specifying step-by-step instructions, Prolog represents knowledge using facts and rules.

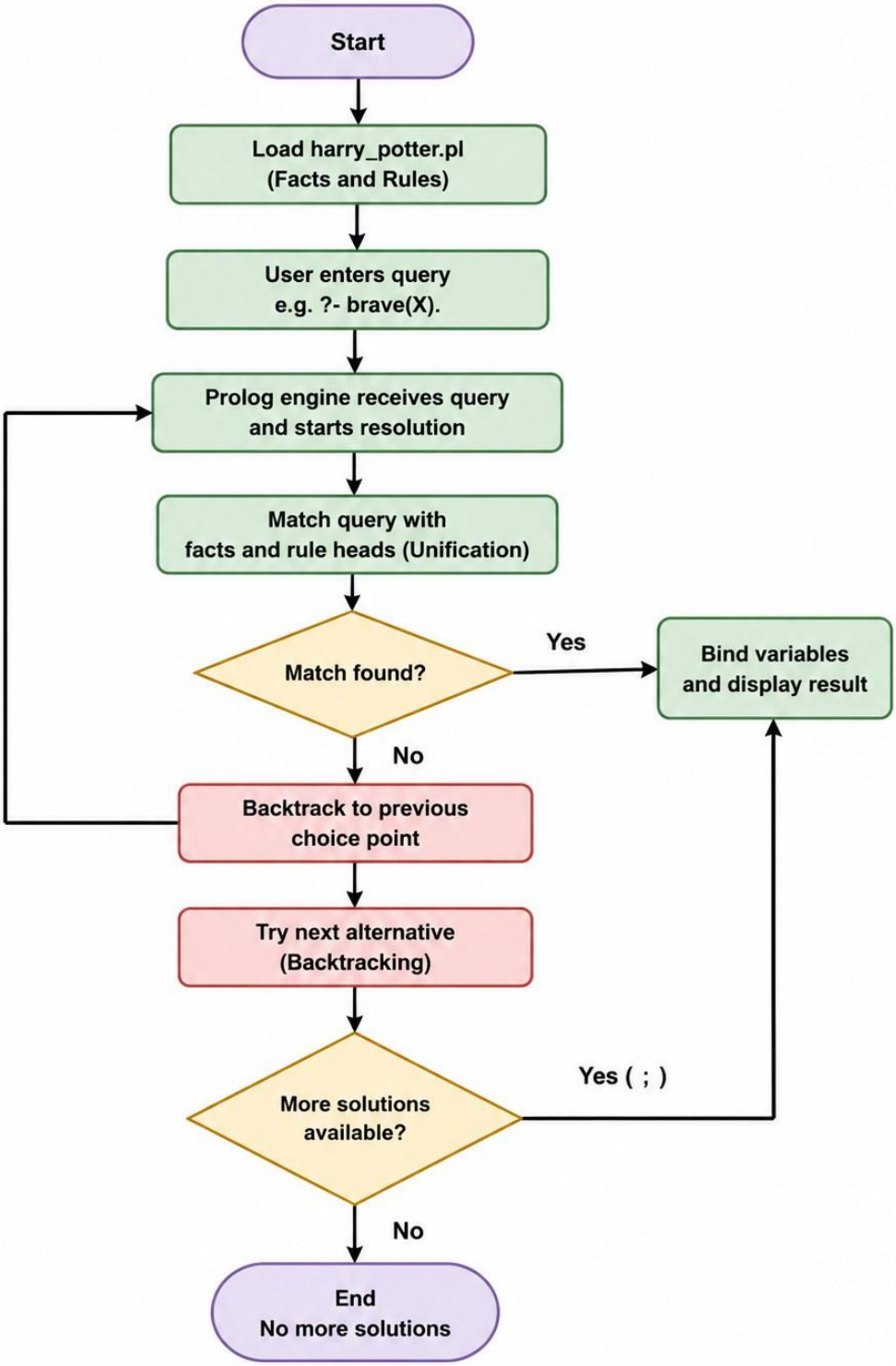
Facts are statements that are directly true, such as house(harry, gryffindor). Rules describe relationships using the form Head :- Body. A query asks Prolog to find information from the knowledge base. Prolog uses unification to match the query with facts and rules and uses backtracking to find additional solutions.

This makes Prolog suitable for small expert systems and rule-based applications because new information can be derived from existing facts without explicitly programming every answer.

PEAS Description

- PEAS stands for Performance measure, Environment, Actuators, and Sensors. For the Harry Potter rule-based advisor:
- Performance Measure: Correct and complete answers to queries and correct rule-based recommendations.
- Environment: The Harry Potter knowledge base containing wizards, houses, skills, objects, situations, and rules.
- Actuators: Prolog output showing answers, true/false results, and alternative solutions.
- Sensors: The query entered by the user, which is matched against the knowledge base.

Flowchart


Start
Load harry_potter.pl
(Facts and Rules)
User enters query
e.g. ?- brave(X).
Prolog engine receives query
and starts resolution
Match query with
facts and rule heads (Unification)
Match found?
Yes
Bind variables
and display result
No
Backtrack to previous
choice point
Try next alternative
(Backtracking)
More solutions
available?
Yes (;)
No
End
No more solutions

Complete Code

```
% Wizards
wizard(harry).
wizard(hermione).
wizard(ron).
wizard(draco).
wizard(luna).

% Houses
house(harry, gryffindor).
house(hermione, gryffindor).
house(ron, gryffindor).
house(draco, slytherin).
house(luna, ravenclaw).

% Skills
skill(harry, defence).
skill(harry, patronus).
skill(hermione, spells).
skill(hermione, potions).
skill(ron, defence).
skill(draco, curses).
skill(luna, creatures).

% Things they have
owns(harry, wand).
owns(harry, cloak).
owns(hermione, wand).
owns(hermione, time_turner).
owns(ron, wand).

% Current problems
situation(harry, dark_creature).
situation(hermione, potion_problem).
situation(ron, locked_door).
situation(draco, duel).
situation(luna, magical_creature).

% Rules
```

```
% Harry can use Patronus for a dark creature
recommended_spell(Wizard, patronus) :-
    situation(Wizard, dark_creature),
    skill(Wizard, patronus).

% A wizard can study a magical creature
recommended_action(Wizard, study_creature) :-
    situation(Wizard, magical_creature),
    skill(Wizard, creatures).

% A wizard with a wand can open a locked door
recommended_spell(Wizard, alohomora) :-
    situation(Wizard, locked_door),
    owns(Wizard, wand).

% Gryffindor wizards are brave
brave(Wizard) :-
    house(Wizard, gryffindor).
```

Sample Queries and Results

1. Find all wizards

```
wizard(X).
```

Result: X = harry ; X = hermione ; X = ron ; X = draco ; X = luna.

2. Find Harry's house

```
house(harry, X).
```

Result: X = gryffindor.

3. Find Hermione's skills

```
skill(hermione, X).
```

Result: X = spells ; X = potions.

4. Find all wizards who know defence

```
skill(X, defence).
```

Result: X = harry ; X = ron.

5. Find what Harry owns

owns(harry, X).

Result: X = wand ; X = cloak.

6. Find Harry's situation

situation(harry, X).

Result: X = dark_creature.

7. Find Harry's recommended spell

recommended_spell(harry, X).

Result: X = patronus.

8. Find Luna's recommended action

recommended_action(luna, X).

Result: X = study_creature.

9. Find who can use Alohomora

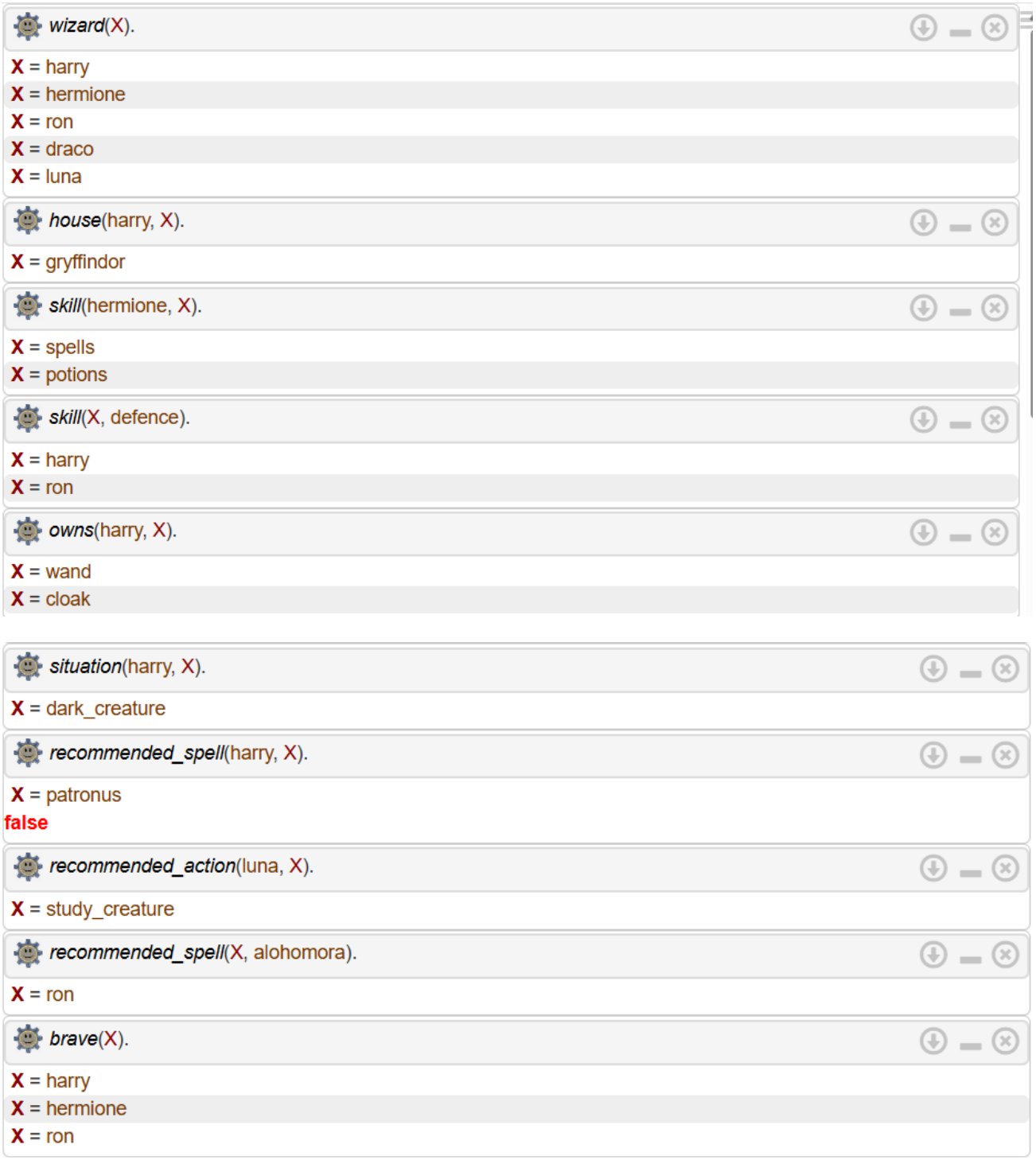
recommended_spell(X, alohomora).

Result: X = ron.

10. Find all brave wizards

brave(X).

Result: X = harry ; X = hermione ; X = ron.



Observation

During execution, it was observed that Prolog searches the facts in the knowledge base and returns matching solutions one at a time. Variables such as X allow Prolog to find unknown values. Rules using commas require all conditions to be satisfied. The recommended_spell, recommended_action, and brave rules derive new information from existing facts rather than storing every answer directly.

Result Analysis

The queries successfully retrieved wizard details, houses, skills, objects, and situations. The rules also correctly derived recommendations and classifications. For example, Harry was identified as

a user of Patronus because his situation is a dark creature and he has the patronus skill. Luna was given the study_creature action because she has creature-related skills. The brave rule correctly identified the Gryffindor wizards.

Result

A Harry Potter themed knowledge-based system was successfully designed and implemented in Prolog. Facts and logical rules were used to represent wizard information and derive new results through queries.

Conclusion

Through this experiment, the basic concepts of Prolog such as facts, rules, variables, queries, unification, and backtracking were studied and applied. The Harry Potter knowledge base demonstrated how a rule-based system can derive useful information from existing facts using simple logical rules.

GITLINK-
https://github.com/prachilohkare8-debug/AIML-202401040025/blob/main/IA_CA/1A/harrypotter.pl